Department of Computer Science and Engineering
Indian Institute of Technology Bombay

CS 236: Operating Systems Lab Spring 2025-26 Labquiz2 (24 marks)	A

Instructions

- No Internet. No phone. No devices. No handwritten notes. No books.
- No friends (strictly for the duration of the exam only!).
- Unethical practices will be an automatic recommendation for the FR grade.
- **Hardcoding of outputs will result in -5 marks per instance.**
- Please read the questions and submission instructions carefully.
- The labquiz consists of 3 problems.
- **All problems are compulsory.**

Submission Guidelines

- **Untar and cd into the labquiz2 directory on your Desktop. Use:**
  - ***./run.sh* to run xv6 in containerized environment**
  - ***submit-xv6* to submit (ask a TA to enter password)**
- **Ensure that your final xv6 submission compiles without any errors. Submissions that do not compile will not be considered for evaluation.**

1. not so lazy sbrk (8 marks)

`sbrklazy(int)` aims to delay allocation of physical memory to valid virtual address ranges until the memory is accessed. As part of this question, implement a configurable version that controls how eagerly the kernel handles lazy allocation of physical memory.

Implement the following system call that takes as an argument the granularity at which the lazy allocation should allocate pages on demand.

```
int sbrklazy_size(int k);
```

The value of `k` indicates the maximum number of pages (at most `k` pages) that the lazy page allocation logic will allocate in one go. On a page fault that needs fixing, the logic will attempt to allocate up to `k` pages **in increasing order of the VA space/range**.

E.g., if the value of `k` is 2, a page fault on a virtual address, will allocate a physical page and add a mapping for this address, and also allocate and map a physical page for the adjoining virtual address page. If the adjoining virtual address range/page is already mapped, it will try to map

the next virtual page and so on, till it finds an unmapped virtual page. This process continues till `k` pages are mapped or till the size of the process.

When `k = 1`, make sure that the lazy page allocation behavior is the same as the default lazy page allocation.

Note that the value of `k` can be changed during the execution of a process.

Implementation of the following system call is already provided — `get_pagefaults()`, which is used to test implementation of `sbrklazy_size(int)`

Sample output

Shell

```
$ p1a 1
Number of pages allocated: 512, k = 1
Expected page faults: 512
Actual page faults: 512
$ p1a 2
Number of pages allocated: 512, k = 2
Expected page faults: 256
Actual page faults: 256
$ p1a 100
Number of pages allocated: 512, k = 100
Expected page faults: 6
Actual page faults: 6
```

Shell

```
$ p1b
sbrklazy_size = 4
Allocated 12 pages lazily

Accessing page 2 | total pgfaults: 1
Accessing page 3 | total pgfaults: 1
Accessing page 5 | total pgfaults: 1
Accessing page 4 | total pgfaults: 1

Accessing page 7 | total pgfaults: 2
Accessing page 8 | total pgfaults: 2
Accessing page 9 | total pgfaults: 2
Accessing page 10 | total pgfaults: 2
```

```
Accessing page 0 | total pgfaults: 3
Accessing page 1 | total pgfaults: 3
Accessing page 6 | total pgfaults: 3
Accessing page 11 | total pgfaults: 3

Accessing page 5 | total pgfaults: 3
Accessing page 12 | usertrap(): unexpected scause 0xf pid=3
            sepc=0x26 stval=0x10000
```

Shell

```
$ p1c
Number of pages allocated: 1024
Accessing 256 pages with sbrklazy_size(1)
Expected page faults: 256
Actual page faults: 256

Accessing 256 pages with sbrklazy_size(2)
Expected page faults: 384
Actual page faults: 384

Accessing 256 pages with sbrklazy_size(4)
Expected page faults: 448
Actual page faults: 448

Accessing 256 pages with sbrklazy_size(1)
Expected page faults: 704
Actual page faults: 704
```

Shell

```
$ p1d
sbrklazy_size = 4
Allocated 12 pages lazily

Accessing page 10 | total pgfaults: 1
Accessing page 11 | total pgfaults: 1
Accessing page 0 | total pgfaults: 2
Accessing page 1 | total pgfaults: 2
```

2. fork + exec = spawn (8 marks)

A call to `fork()`, in a lot of cases, is immediately followed by a call to `exec()` to start a different binary. Such an approach creates a lot of overhead of copying memory and context switches. An alternate approach is to use a `spawn()` system call to create the process with the new binary, resulting in a significant speedup. Implement this system call with the following signature (the same as `exec`):

```
C/C++
int spawn(char *path, char **argv);
```

`spawn` should return the PID of the child process if it is successful, and -1 if it fails.
Programs `p2a.c`, `p2b.c` and `p2c.c` have been provided for you to test your implementation.

If `spawn` is invoked as `spawn(0, 0)`, then make it behave like `fork()`.

Sample output:

```
Shell
$ p2a p2b 10 hello
[pid 3] Parent: spawning child p2b with pid 4
[pid 4] Child: hello
[pid 3] Parent: child p2b with pid 4 terminated with status 10
```

```
Shell
$ p2c
[pid 3] Parent: Calling spawn(0, 0)
[pid 4] Child: hello.
[pid 3] Parent: goodbye. Reaped process with pid 4.
```

Hints:
Fork does the following:

1. Allocates a new pcb.
2. Copies page tables.
3. Copies trapframe.
4. Copies file descriptors table.
5. Sets the cwd (current working directory), parent pid and run state of the new process.

Exec does the following:

1. Opens and reads the new file.
2. Copies file content using page table pointer of process.
3. Creates a new pagetable, allocates memory for the new file in the pagetable, loads the program segment into pagetable.
4. Allocates stack for the process, sets up a trapframe.
5. Frees the old pagetable and sets `p->pagetable` to the new pagetable.

`spawn` will have to combine the functionalities of both `fork` and `exec` without copying the pagetable from the old process. You can safely use the code of the xv6 functions about file copying, trapframe setup, etc. as long as you are consistent with it.

3. fast! … up time. (8 marks)

Some operating systems (e.g., Linux) speed up certain system calls by sharing data in a read-only region between userspace and the kernel. This eliminates the need for kernel crossings when performing these system calls.

Can the syscall overhead for retrieving system **uptime** be eliminated? A possible solution —

- Allocate a single kernel virtual page. You need not to map this address at a particular VA since you can just store the kernel VA in a global variable for reusability. Use the global variable to access the page in the kernel for updates.
- Map this kernel page into every process's user virtual address space at user virtual address `FASTUPTIME_PAGE`. Make sure you unmap this address when the process exits.
- Update the value of `ticks` in this shared page as part of the clock interrupt handler.
- Expose an userspace function `fastuptime()` that directly reads from this mapped page (Lookup `sbrklazy` for how to add a userspace function).

```
C/C++
/*map the fastuptime_page to the following user va in process pagetables*/
#define FASTUPTIME_PAGE (TRAPFRAME - PGSIZE)
```

Note that only one physical page is allocated in this problem.

Hints:

1. The kernel page can be set up as part of the `kvmmake` function.
2. Make sure that the user space process can only read, while the kernel can read and write to the shared page.
3. Make sure you unmap the user virtual address `FASTUPTIME_PAGE` when the process exits.

Sample output:

Test if the page can only be read and NOT be written from a user space process.

```
Shell
$ p3a
Reading addr FASTUPTIME_PAGE (0x0000003FFFFFD000)
Expecting to not crash on read
Read value: 14
Writing to addr FASTUPTIME_PAGE (0x0000003FFFFFD000)
Expecting to crash on write
usertrap(): unexpected scause 0xf pid=3
            sepc=0x44 stval=0x3fffffd000
```

Test if all the user space processes map to the same physical address for the virtual address of the shared page used for **`fastuptime()`** function call.

```
Shell
$ p3b
Usage: p3b <num_processes>
$ p3b 100
Creating 100 processes...
Passed: FASTUPTIME_PAGE has the same PA in all child processes.
```

Test if the **`fastuptime()`** is actually correct (gives a timestamp close to **`uptime()`**) and it does give a speed up as compared to the **`uptime()`** system call.

```
Shell
$ p3c
Usage: p3c <iterations>
$ p3c 100000
start: uptime() = 67    fastuptime() = 67
fastuptime() * 100000: 0 ticks
uptime()     * 100000: 19 ticks
end  : uptime() = 86    fastuptime() = 86
$ p3c 1000000
start: uptime() = 148   fastuptime() = 148
fastuptime() * 1000000: 1 ticks
uptime()     * 1000000: 196 ticks
end  : uptime() = 345   fastuptime() = 345
```

Test if the **`fastuptime()`** function call works for multiple processes correctly.

Shell

```
$ p3d
Usage: p3d <num_processes>
$ p3d 3
[Parent] Creating 3 processes...
[Parent] uptime() = 42  fastuptime() = 42
[Parent] Child-1 will sleep for 4 ticks
[Parent] Child-2 will sleep for 8 ticks
[Parent] Child-3 will sleep for 12 ticks
[Child 1] woke up! uptime() = 46        fastuptime() = 46
[Child 2] woke up! uptime() = 50        fastuptime() = 50
[Child 3] woke up! uptime() = 54        fastuptime() = 54
```